INTERNSHIP REPORT

PFE - Final Year Project

SPRINT 4

Debugging Pop Quiz

Full Name : Ayoub Barahal

Duration : the weekend

Status : 100% Completed

"Debugging, performance optimization and web security mastery."

Table des matières

1 Introduction **2**

2 Environment Setup **2**
2.1 Backend 2
2.2 Frontend 2

3 Backend Challenges **2**
3.1 Error Handling – Endpoint /broken 2
3.1.1 Situation 2
3.1.2 Analysis 2
3.1.3 Solution Implemented 2
3.2 Simultaneous Connections – Endpoint /crash 3
3.2.1 Situation 3
3.2.2 Analysis 4
3.2.3 Solution Implemented 4
3.2.4 new code 4
3.2.5 Apache Benchmark After Fix 5

4 Frontend Challenges **7**
4.1 Fetch / XHR Issue – /fetch 7
4.1.1 Situation 7
4.1.2 Analysis 7
4.1.3 Solution Implemented 7
4.1.4 Result After Fix 8
4.2 XHR Issue – /users 8
4.2.1 3.3.1 Situation 8
4.2.2 3.3.2 Analysis 8
4.2.3 Solution Implemented 9
4.2.4 3.3.4 Result After Fix 9
4.3 Assets Optimization 10
4.3.1 Problem 10
4.3.2 Solution 10
4.4 XSS Vulnerability – /security 10
4.4.1 Problem 10
4.4.2 Security Fix – CSP 10

5 Conclusion **11**

1 Introduction

Sprint 4 evaluated backend and frontend debugging skills. The objective was to identify production-level issues related to errors, performance bottlenecks, caching and security, and implement professional solutions.

2 Environment Setup

2.1 Backend

- cd backend
- composer install
- php -S localhost :8888

2.2 Frontend

- cd frontend
- cp .env.example .env
- npm install
- npm run dev

3 Backend Challenges

3.1 Error Handling – Endpoint /broken

3.1.1 Situation

- The endpoint `http://localhost:8888/broken` returns an HTTP 500 error.
- No clear error message is displayed.

3.1.2 Analysis

- HTTP 500 means **Internal Server Error**.
- The route `/broken` performs the same logic as the root route.

After investigation, the issue was caused by characters written using `write()` that were not UTF-8 compatible.

3.1.3 Solution Implemented

All text output is converted to UTF-8 before writing :

This ensures proper encoding compatibility and prevents internal server errors.

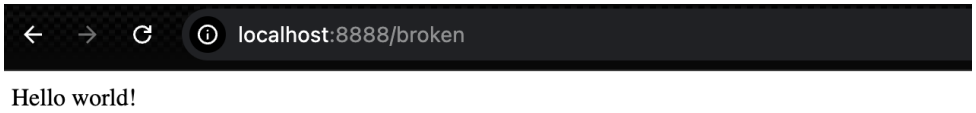


FIGURE 1 – screen of the endpoint /broken

3.2 Simultaneous Connections – Endpoint /crash

3.2.1 Situation

Apache Benchmark test executed :

```
ab -n 200 -c 10 http://localhost:8888/crash
```

Observations :

```
Benchmarking localhost (be patient)
Completed 100 requests
Completed 200 requests
Finished 200 requests


Server Software:
Server Hostname:        localhost
Server Port:            8888

Document Path:          /crash
Document Length:        0 bytes

Concurrency Level:      10
Time taken for tests:   0.248 seconds
Complete requests:      200
Failed requests:        0
Non-2xx responses:      200
Total transferred:      70000 bytes
HTML transferred:       0 bytes
Requests per second:    806.38 [#/sec] (mean)
Time per request:       12.401 [ms] (mean)
Time per request:       1.240 [ms] (mean, across all concurrent requests)
Transfer rate:          275.62 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    0   0.0      0       0
Processing:     2   12   1.8     12      16
Waiting:        2   12   1.9     12      16
Total:          2   12   1.8     12      16

Percentage of the requests served within a certain time (ms)
  50%     12
  66%     12
  75%     13
  80%     13
  90%     14
  95%     15
  98%     16
  99%     16
```

FIGURE 2 – screen of the endpoint /broken

3.2.2 Analysis

The function `logRequestWithRotation` originally :

- Read the entire log file at every request.
- Wrote a massive block of characters when exceeding 10 lines.
- Appended new data without file locking.

Identified problems :

- Full file reading → high memory usage.
- Large write operations → server blocking.
- No synchronization → risk of concurrent write conflicts.

3.2.3 Solution Implemented

3.2.4 new code

```
1 reference
function logRequestWithRotation($message): void
{
    $logFile = __DIR__ . '/request_log.txt';
    $logEntry = "[" . date(format: "Y-m-d H:i:s") . "] " . $message . PHP_EOL;

    // Vérifier la taille du fichier (max 1 Mo)
    if (file_exists(filename: $logFile) && filesize(filename: $logFile) > 1024 * 1024) {
        file_put_contents(filename: $logFile, data: ""); // vider le fichier
    }

    // Écriture sûre
    file_put_contents(filename: $logFile, data: $logEntry, flags: FILE_APPEND | LOCK_EX);
}
```

FIGURE 3 – new code

3.2.5 Apache Benchmark After Fix

`ab -n 200 -c 10 http://localhost:8888/crash`

- Requests processed quickly
- No crash or slowdown.
- Log system remains reliable and secure.

```
→  ~ ab -n 200 -c 10 http://localhost:8888/crash
This is ApacheBench, Version 2.3 <$Revision: 1913912 $>
Copyright 1996 Adam Twiss, Zeus Technology Ltd, http://www.zeustech.net/
Licensed to The Apache Software Foundation, http://www.apache.org/

Benchmarking localhost (be patient)
Completed 100 requests
Completed 200 requests
Finished 200 requests


Server Software:
Server Hostname:        localhost
Server Port:            8888

Document Path:          /crash
Document Length:        12 bytes

Concurrency Level:      10
Time taken for tests:   0.225 seconds
Complete requests:      200
Failed requests:        0
Total transferred:      68600 bytes
HTML transferred:       2400 bytes
Requests per second:    890.42 [#/sec] (mean)
Time per request:       11.231 [ms] (mean)
Time per request:       1.123 [ms] (mean, across all concurrent requests)
Transfer rate:          298.25 [Kbytes/sec] received

Connection Times (ms)
              min  mean[+/-sd] median   max
Connect:        0    0   0.1      0       0
Processing:     2   11   2.0     10      17
Waiting:        2   11   2.0     10      17
Total:          3   11   2.0     10      18

Percentage of the requests served within a certain time (ms)
  50%     10
  66%     11
  75%     12
  80%     12
```

FIGURE 4 – Apache Benchmark after Fix

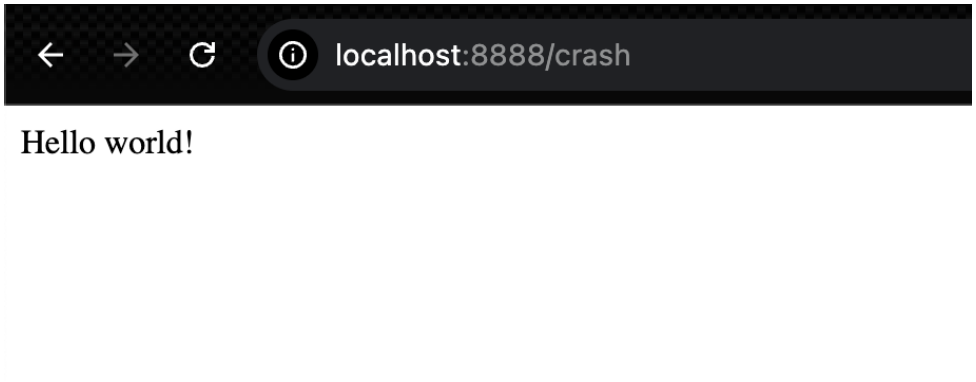


FIGURE 5 – Apache Benchmark after Fix

4 Frontend Challenges

4.1 Fetch / XHR Issue – /fetch

4.1.1 Situation

- The endpoint `http://localhost:8888/fetch` returns an **HTTP 401 error**.
- The request does not succeed when called from the frontend (Fetch/XHR).

4.1.2 Analysis

- HTTP 401 means **Unauthorized** — authentication is required.
- The `/fetch` endpoint is protected with **Basic Authentication**.
- The frontend request was sent without the `Authorization` header, so the server rejected it.

Additionally :

- There was a **CORS** issue (Cross-Origin Resource Sharing).
- The backend did not properly allow the `Authorization` header.
- The browser blocked the request before it was processed correctly.

So two problems were identified :

1. Missing Basic Authentication credentials → 401 error.
2. CORS configuration not allowing the Authorization header.

4.1.3 Solution Implemented

```php
// middleware CORS — gère aussi les requêtes preflight OPTIONS
$app->add(function ($request, $handler): mixed {
    $method = strtoupper(string: $request->getMethod());

    if ($method === 'OPTIONS') {
        $resp = new \Slim\Psr7\Response();
        return $resp
            ->withHeader('Access-Control-Allow-Origin', 'http://localhost:5173')
            ->withHeader('Access-Control-Allow-Methods', 'GET, POST, PUT, DELETE, OPTIONS')
            ->withHeader('Access-Control-Allow-Headers', 'Content-Type, Authorization')
            ->withHeader('Access-Control-Allow-Credentials', 'true');
    }

    $response = $handler->handle($request);
    return $response
        ->withHeader('Access-Control-Allow-Origin', 'http://localhost:5173')
        ->withHeader('Access-Control-Allow-Methods', 'GET, POST, PUT, DELETE, OPTIONS')
        ->withHeader('Access-Control-Allow-Headers', 'Content-Type, Authorization')
        ->withHeader('Access-Control-Allow-Credentials', 'true');
});
```

FIGURE 6 – index.php

```
function Fetch() {
  useEffect(() => {

    const username = "username";
    const password = "password";        You, 19 hours ago • Uncommitted changes

    const credentials = btoa(`${username}:${password}`);

    fetch(`${import.meta.env.VITE_API_URL}/fetch`, {
      headers: {
        Authorization: `Basic ${credentials}`,
      },
    })
      .then((response) => {
        if (!response.ok) {
          throw new Error(`Erreur API: ${response.status}`);
        }
        return response.json();
      })
      .then((data) => console.log("Réponse backend:", data))
      .catch((err) => console.error(err));
  }, []);

  return <div className="p-2">Hello from fetch!</div>;
}
```

FIGURE 7 – index.php

4.1.4 Result After Fix

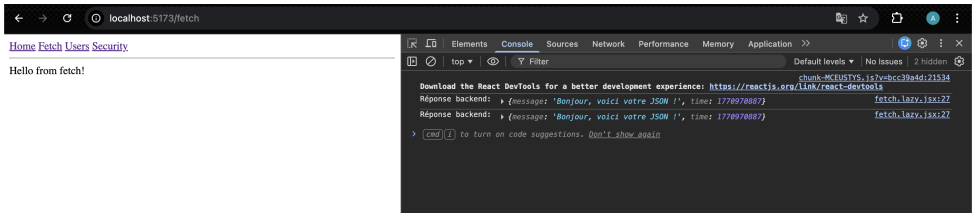


FIGURE 8 – /fetch after fix

4.2 XHR Issue – /users

4.2.1 3.3.1 Situation

— The endpoint `http://localhost:8888/users` requires a `GET` request.
— Sending a `POST` request returns **HTTP 405 Method Not Allowed**.
— The frontend requests were initially using the wrong method, causing errors.

4.2.2 3.3.2 Analysis

— The route is defined as follows in the backend :

```php
$app->any('/users', function (Request $request, Response $response, $args): mixed {
    if ($request->getMethod() === 'POST') {
        $response->getBody()->write("Method Not Allowed");
        return $response
            ->withStatus(405);
    }

    $utilisateurs = array(
        array(
            'id' => 1,
            'nom' => 'Jean Dupont',
            'email' => 'jean.dupont@example.com',
            'role' => 'administrateur'
        ),
```

FIGURE 9 – index.php

— Only `GET` requests are allowed.
— The frontend needed to modify the request method to `GET`.

4.2.3 Solution Implemented

Frontend Fix : Use GET Request

```jsx
function Fetch() {
  useEffect(() => {
    fetch(`${import.meta.env.VITE_API_URL}/users`, {
      method: "GET",      You, 20 hours ago • Uncommitted changes
    })
      .then((response) => response.json())
      .then((data) => console.log(data));
  }, []);

  return <div className="p-2">Hello from users!</div>;
}
```

FIGURE 10 – index.php

4.2.4 3.3.4 Result After Fix

— The request now uses `GET`, matching backend requirements.
— The endpoint responds with the JSON list of users.
— HTTP 405 errors are resolved.
— Frontend can now display the user data correctly.

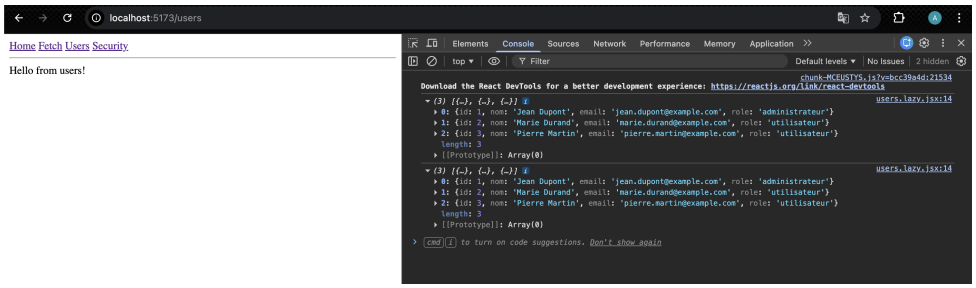


FIGURE 11 – /users after fix

4.3 Assets Optimization

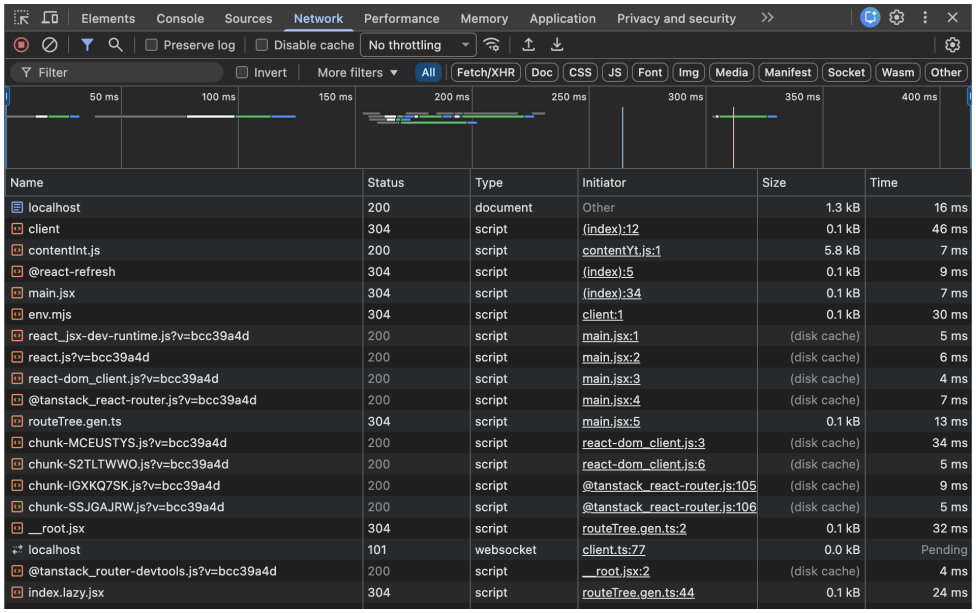
4.3.1 Problem

Assets downloaded on every page reload.

4.3.2 Solution

- Implemented Cache-Control headers
- Enabled asset versioning (hashing)
- Used long cache duration for static assets
- Allowed JS cache refresh when modified

Result : Browser returned 304 Not Modified and caching worked properly.



Name	Status	Type	Initiator	Size	Time
localhost	200	document	Other	1.3 kB	16 ms
client	304	script	(index):12	0.1 kB	46 ms
contentInt.js	200	script	contentYt.js:1	5.8 kB	7 ms
@react-refresh	304	script	(index):5	0.1 kB	9 ms
main.jsx	304	script	(index):34	0.1 kB	7 ms
env.mjs	304	script	client:1	0.1 kB	30 ms
react_jsx-dev-runtime.js?v=bcc39a4d	200	script	main.jsx:1	(disk cache)	5 ms
react.js?v=bcc39a4d	200	script	main.jsx:2	(disk cache)	6 ms
react-dom_client.js?v=bcc39a4d	200	script	main.jsx:3	(disk cache)	4 ms
@tanstack_react-router.js?v=bcc39a4d	200	script	main.jsx:4	(disk cache)	7 ms
routeTree.gen.ts	304	script	main.jsx:5	0.1 kB	13 ms
chunk-MCEUSTYS.js?v=bcc39a4d	200	script	react-dom_client.js:3	(disk cache)	34 ms
chunk-S2TLTWWO.js?v=bcc39a4d	200	script	react-dom_client.js:6	(disk cache)	5 ms
chunk-IGXKQ7SK.js?v=bcc39a4d	200	script	@tanstack_react-router.js:105	(disk cache)	9 ms
chunk-SSJGAJRW.js?v=bcc39a4d	200	script	@tanstack_react-router.js:106	(disk cache)	5 ms
__root.jsx	304	script	routeTree.gen.ts:2	0.1 kB	32 ms
localhost	101	websocket	client.ts:77	0.0 kB	Pending
@tanstack_router-devtools.js?v=bcc39a4d	200	script	__root.jsx:2	(disk cache)	4 ms
index.lazy.jsx	304	script	routeTree.gen.ts:44	0.1 kB	24 ms

FIGURE 12 – activate the cache

4.4 XSS Vulnerability – /security

4.4.1 Problem

Image loaded from unknown external source.

4.4.2 Security Fix – CSP

- default-src 'self'
- img-src 'self'
- script-src 'self'
- style-src 'self'

Result : External malicious resources blocked.

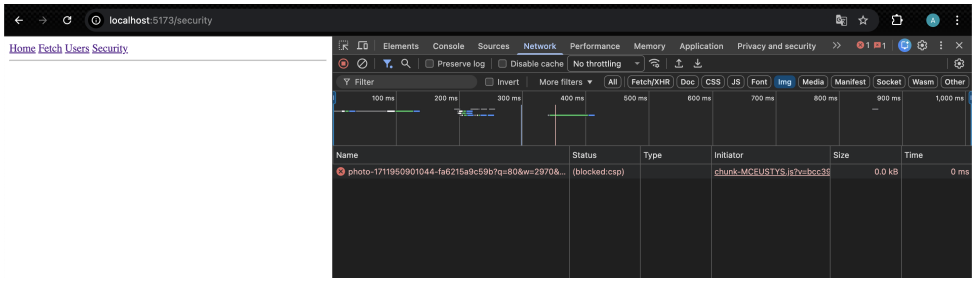


FIGURE 13 – index.php

5 Conclusion

Sprint 4 reinforced :

- Backend debugging and log analysis
- Performance optimization under load
- CORS and API configuration
- Browser caching strategy
- XSS protection with Content Security Policy

This sprint simulated real production challenges and strengthened professional debugging and security skills.